

PLATFORM ENGINEERING · CAPSTONE

Reference Architecture

How the platform is deployed — a Terraform baseline that hands off to Argo CD, and how workloads are segregated across resource planes.

kind cluster

Terraform → Argo CD

App-of-apps GitOps

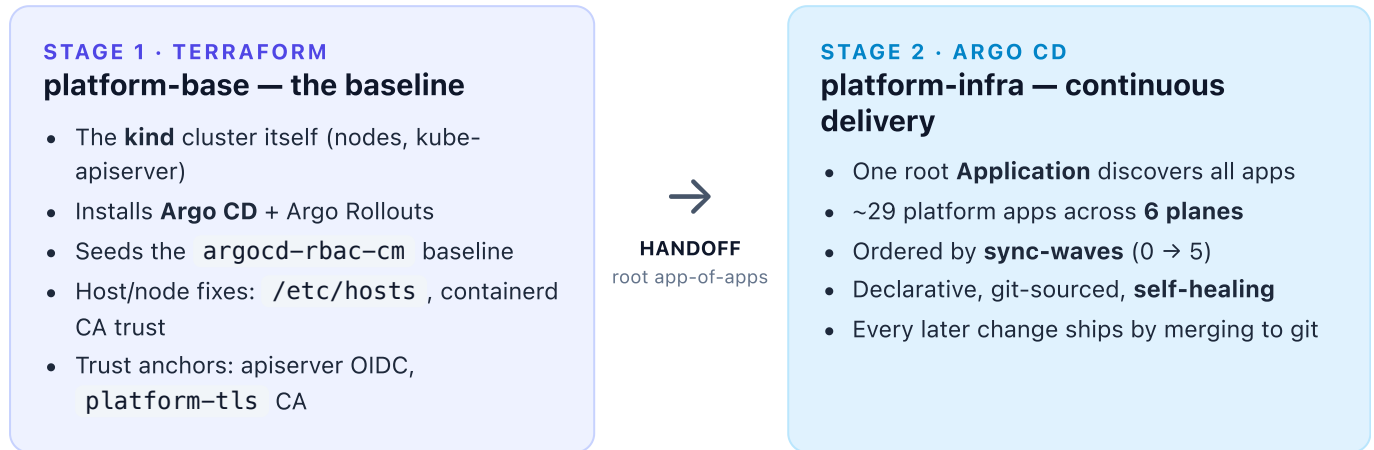
6 planes · AppProject isolation

Repositories platform-base (Terraform) · platform-infra (GitOps source of truth) · platform-idp (app source)

Scope Cluster bring-up, the Terraform→GitOps handoff, and plane segregation of the ~29 platform Applications.

Two tools, one boundary

The platform is stood up in two stages by two tools with a single, deliberate boundary between them. Terraform builds the **baseline** — the things that must exist before any in-cluster GitOps can run. It then hands control to **Argo CD**, which continuously reconciles everything else from git.



The boundary rule. If something **must exist before Argo CD can run**, or lives at the **cluster/host lifecycle** level, Terraform owns it. If it is an **in-cluster workload or config** that benefits from continuous reconciliation against git, Argo CD owns it. The two never fight over the same object.

Why not one tool for everything?

Why Terraform for the baseline

- **Chicken-and-egg:** Argo CD cannot install Argo CD — something outside the cluster must create the cluster and the GitOps engine.
- **Host & cluster lifecycle:** nodes, apiserver flags, node `/etc/hosts` and containerd trust are not in-cluster manifests.
- **One-time trust anchors** (CA, RBAC baseline) that GitOps then builds on.

Why Argo CD for the rest

- **Continuous reconciliation:** drift is corrected automatically (self-heal), not only at `apply` time.
- **Git as source of truth:** every change is a reviewed commit; the cluster converges to it.
- **Ordering & health gates** across dozens of apps via sync-waves — hard to express in a flat Terraform graph.

platform-base — the Terraform baseline

A single `terraform apply` from the operator's machine provisions the cluster and everything the GitOps layer depends on. Each concern is its own file, so the baseline is auditable and reproducible.

Terraform file	Provisions & why it's in the baseline
<code>main.tf</code>	The kind cluster — control-plane + worker nodes. Nothing else can exist first.
<code>argocd.tf</code>	Installs Argo CD and seeds the <code>argocd-rbac-cm</code> static baseline. This is the GitOps engine — it cannot deploy itself.
<code>rollouts.tf</code>	Argo Rollouts controller + CRDs, so tenant workloads can use progressive delivery from day one.
<code>apiserver-oidc.tf</code>	kube-apiserver <code>--oidc=*</code> flags + <code>oidc-ca.crt</code> so <code>kubectl</code> logs in via Keycloak. An apiserver (host) concern, not a manifest.
<code>platform-tls.tf</code>	The platform-tls wildcard CA — the trust anchor every ingress and OIDC login chains to. Must be a CA certificate.
<code>node-hosts.tf</code>	Installs a systemd unit on each node so <code>*.sslip.io</code> names resolve on boot (Harbor pulls, apiserver→Keycloak). Node-level, survives restarts.
<code>node-containerd-trust.tf</code>	Distributes the <code>platform-tls</code> CA into each node's containerd trust store so image pulls from Harbor verify.
<code>registry.tf</code>	Registry wiring for the in-cluster image path.
<code>providers.tf</code> · <code>versions.tf</code> · <code>variables.tf</code> · <code>outputs.tf</code>	Provider config, version pins, inputs, and outputs (e.g. kubeconfig) consumed by the bootstrap.

One deliberate exception — OpenBao runtime state. A fresh cluster re-initialises OpenBao, whose unseal/init state and OIDC bootstrap are **not** reproduced by `terraform apply`. This is the single documented manual post-step (see `bootstrap/README.md`); everything else returns automatically.

From imperative baseline to declarative delivery

The baseline stage concludes by handing the cluster to Argo CD. Terraform's last contributions are the GitOps engine and its endpoint; from that moment Argo CD — not Terraform — owns the running platform.

HOST / KIND	Nodes, kube-apiserver, containerd, node /etc/hosts — created & patched by Terraform
BASELINE	Argo CD · Argo Rollouts · argocd-rbac-cm · platform-tls CA · apiserver OIDC — installed by Terraform
HANDOFF	Plane AppProjects + the root app-of-apps Application (bootstrap/root-app.yaml) — the endpoint Argo CD reconciles
PLATFORM	~29 Applications across 6 planes — reconciled continuously from platform-infra/apps/

How the root app-of-apps works

A single root Application points at apps/ with **recursive directory** discovery, including only Application manifests (**/application.yaml , **/*-application.yaml). Argo CD renders one **child Application per platform app** — so adding an app is just committing a new application.yaml ; the root picks it up on the next sync. Grouping into planes is expressed by directory (apps/<plane>/<app>/), a plane label, and one AppProject per plane.

Why a single recursive root (not a per-plane hierarchy). Sync-waves only order apps **within one parent's** sync. Keeping every leaf under one root means the wave ordering holds **across planes** — e.g. wave 0 spans the resource, security and monitoring planes together, and Argo CD holds wave 1 until every wave-0 app is **Synced and Healthy**.

After the handoff

Argo CD runs with **automated sync + self-heal**. Manual kubectl edits to managed objects are reverted to the git-declared state — the cluster always converges to platform-infra's main . Terraform is only re-run for **baseline** changes (cluster, nodes, the engine itself), never for app delivery.

Resource planes of platform engineering

Platform apps are segregated into **planes** — each a directory under `apps/` and an Argo CD **AppProject**. The AppProject is the guardrail: it constrains which repo a plane's apps may source from, which destinations they may deploy to, and which resource kinds they may touch. Planes give blast-radius containment, clear ownership, and cross-plane wave ordering.

resource

6 apps

Foundational cluster capabilities everything else builds on.

ingress-nginx cloudnative-pg keycloak-db
crossplane crossplane-providers
tenant-priority-classes

security

12 apps

Identity, secrets, admission policy, supply-chain & runtime security.

keycloak openbao spire spire-crds
cert-manager gatekeeper tenant-guardrails
image-policy ratify trivy-operator
trivy-gatekeeper falco

integration-delivery

2 apps

The image registry and its replication pipeline.

harbor harbor-replication

developer-control

4 apps

The self-service internal developer platform (portal, API, controller).

teams-app teams-api teams-operator
teams-operator-argocd-rbac

monitoring

2 apps

Cluster metrics and the observability stack.

metrics-server monitoring-stack

tenant-workloads

AppProject

Isolation boundary for demo/tenant apps — kept separate from platform planes.

demo-* tenant applications

Plane = directory + label + AppProject. The directory `apps/<plane>/` organises source; the `plane` label makes membership queryable; the AppProject in `bootstrap/projects/<plane>.yaml` enforces the guardrails. A plane's apps can only source from `platform-infra`, keeping the supply chain narrow.

The deployment sequence

From an empty machine to a converged platform, in order.

- 1 **terraform apply** (platform-base) creates the kind cluster — control-plane and worker nodes.
- 2 Terraform installs **Argo CD** and **Argo Rollouts**, seeds the `argocd-rbac-cm` baseline, applies apiserver OIDC, the `platform-tls` CA, and the node `/etc/hosts` + containerd-trust fixes.
- 3 The bootstrap applies the **plane AppProjects** and the **root app-of-apps** (`bootstrap/root-app.yaml`) — the handoff point.
- 4 The root Application **recursively discovers** every `application.yaml` and creates one child Application per platform app.
- 5 Argo CD reconciles by **sync-wave**, holding each wave until the previous is **Synced & Healthy**.
- 6 The platform **converges**; self-heal keeps it there. Thereafter, changes ship by merging to `platform-infra`.
- 7 **One manual post-step**: initialise / unseal OpenBao and run the OIDC SSO bootstrap (per `bootstrap/README.md`).

Sync-wave ordering (the dependency spine)

Waves run low-to-high across all planes at once. Foundations come first; anything that depends on them comes later.



Illustrative grouping — the authoritative wave for each app is the `argocd.argoproj.io/sync-wave` annotation on its Application. The principle is invariant: a dependent app never syncs before the foundation it needs is Healthy.